

2.3 解: 该问题是 0-1 背包问题.

目标函数: $\max \sum_{i=1}^n v_i x_i, x_i = 0, 1$

约束条件: $\sum_{i=1}^n l_i x_i \leq D, x_i = 0, 1$

① 算法描述:

优化函数: $W_k(l)$, 表示在前 k 个货柜中选择且总的底面长度不超过 l 时的最大仓储收益.

标记函数: $I_k(l)$, 表示在找到 $W_k(l)$ 的情况下选择的货柜的最大编号.

递推关系: $W_k(l) = \max \{ W_{k-1}(l), W_{k-1}(l-l_k) + v_k \}$
 $k \in \{1, 2, \dots, n\}, l \in (-\infty, D]$
 $I_k(l) = \begin{cases} I_{k-1}(l), & W_{k-1}(l) > W_{k-1}(l-l_k) + v_k \\ k, & W_{k-1}(l) \leq W_{k-1}(l-l_k) + v_k \end{cases}$

初值: $\forall k \in \{1, 2, \dots, n\}, l < 0, W_k(l) = -\infty$

$\forall k \in \{1, 2, \dots, n\}, W_k(0) = 0$

$W_k(l) = \begin{cases} v_k, & l \geq l_k \\ 0, & 0 \leq l < l_k \end{cases}, I_k(l) = \begin{cases} 1, & l \geq l_k \\ 0, & 0 \leq l < l_k \end{cases}$

由递推关系可知: 满足子问题重叠性与优化原则.

② 伪代码

Max Rent

输入: $L = [l_1, l_2, \dots, l_n], n, D, v$

输出: W, I

for $i \leftarrow 1$ to n do

$W[i][0] \leftarrow 0$

for $j \leftarrow 1$ to D do

if $j \geq L[i]$ then

$W[i][j] \leftarrow v[i]$

else

$W[i][j] \leftarrow 0$

for $k \leftarrow 2$ to n do

for $l \leftarrow 1$ to D do

$tmp \leftarrow 0$

if $l \geq L[k]$ then

$tmp \leftarrow W[k-1][l-L[k]] + v[k]$

$W[k][l] \leftarrow \max \{ W[k-1][l], tmp \}$

if $W[k][l] = W[k-1][l]$ then

$I[k][l] = I[k-1][l]$

else

$I[k][l] = k$

Track Solution

输入: I

输出: 货柜的选择方案 $x_1, x_2, \dots, x_n$

for $i \leftarrow 1$ to n

$x_i \leftarrow 0$

while $j > 1$ do:

$j \leftarrow I[n][D]$

$x_j \leftarrow 1$

$n \leftarrow j-1$

$D \leftarrow D - l_j$

时间复杂度为 $O(nD)$

2.5 解: 该问题类似于背包问题

① 算法描述

优化函数: $m[i, j]$ 表示用前 i 种硬币支付 j 元的最小硬币重量

标记函数: $t[i, j]$ 记录 $m[i, j]$ 取得最小时选择的硬币的最大编号

递推关系: $m[i, j] = \min \{ m[i-1, j], m[i, j-v_i] + w_i \}$

$0 \leq i \leq n, 0 \leq j \leq y$

$t[i, j] = \begin{cases} t[i-1, j], & m[i-1, j] \leq m[i, j-v_i] + w_i \\ i, & m[i-1, j] > m[i, j-v_i] + w_i \end{cases}$

初值: $m[i, 0] = 0, m[i, x] = +\infty, x < 0$

$m[1, x] = \lfloor \frac{x}{v_1} \rfloor w_1, 0 \leq x \leq y$

$t[1, x] = 1, 0 \leq x \leq y$

$t[i, 0] = 0$

由递推关系可知: 满足子问题重叠性与优化原则.

② Coin

输入: $w[1, 2, \dots, n], v[1, 2, \dots, n], y$

输出: m, t

for $k \leftarrow 1$ to y do

$m[1, k] \leftarrow k / v_1 * w_1$

$t[1, k] \leftarrow 1$

for $i \leftarrow 2$ to n do

for $j \leftarrow 1$ to y do

$m[i, j] \leftarrow m[i-1, j]$

$t[i, j] \leftarrow t[i-1, j]$

if $j-v[i] \geq 0$ and $m[i, j-v[i]] + w[i] < m[i-1, j]$ then

$m[i, j] \leftarrow m[i, j-v[i]] + w[i]$

$t[i, j] \leftarrow i$

Track Solution

输入: t, v

输出: 使用每种硬币的个数 $m_1, m_2, \dots, m_n$

1. $i \leftarrow n, j \leftarrow y$
2. $i \leftarrow t[i, j]$
3. $m_i \leftarrow 1$
4. $j \leftarrow j - v[i]$
5. while $t[i, j] = i$ do
6. $m_i \leftarrow m_i + 1$
7. $j \leftarrow j - v[i]$
8. if $t[i, j] \neq 0$ then
9. goto 2

③ 时间复杂度: $O(ny)$

④ 输入实例: $v_1=1, v_2=4, v_3=6, v_4=8, y=12$
 $w_1=1, w_2=2, w_3=4, w_4=6$

$m[i, j] / t[i, j]$	j	1	2	3	4
1	1	1	1	1	1
2	2	2	2	2	2
3	3	3	3	3	3
4	4	2	2	2	2
5	5	3	3	3	3
6	6	4	4	4	4
7	7	5	5	5	5
8	8	4	4	4	4
9	9	5	5	5	5
10	10	6	6	6	6
11	11	7	7	7	7
12	12	6	6	6	6

$$m[4, 12] = 6$$

$$t[4, 12] = 2 \Rightarrow t[2, 8] = 2 \Rightarrow t[2, 4] = 2 \Rightarrow t[2, 0] = 0$$

故用了 3 枚面值为 4 的硬币。

3.7 解: (1) 目标函数: $\max \sum_{i=1}^n p_i k_i, k_i \in \{0, 1\}$

约束条件: $\forall i, j \in \{1, 2, \dots, n\}, i \neq j, k_i + k_j = 1$
 $\Rightarrow |x_i - x_j| \geq d$

(2) 算法描述:

将这 n 个位置排序后放置在坐标轴上 (从原点开始放置)

设坐标依次为 $x_1, x_2, \dots, x_n (x_1 < x_2 < \dots < x_n)$

优化函数: $M[k, x]$ 表示考虑前 k 个位置, 坐标值不超过 x 的
 最大总收益。

标记函数: $t[k, x]$ 表示当 $M[k, x]$ 取到最大值时
 的最远位置

递推关系: $M[k, x] = \max \{ M[k-1, x], M[k-1, x-d] + p_k \}$
 $t[k, x] = \begin{cases} k, & M[k-1, x] < M[k-1, x-d] + p_k \\ t[k-1, x], & M[k-1, x] \geq M[k-1, x-d] + p_k \end{cases}$

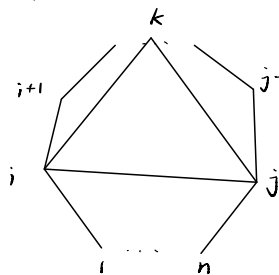
初值: $M[1, x] = \begin{cases} p_1, & x \geq x_1 \\ 0, & x < x_1 \end{cases}$
 $t[1, x] = \begin{cases} 1, & x \geq x_1 \\ 0, & x < x_1 \end{cases}$

③ 时间复杂度:

$$O(n \log n) + O(n \log n)$$

$$= O(n \log n + n \log n)$$

3.12 解: 如图所示, 考虑有 n 个顶点的凸多边形



优化函数: $m[i, j]$ 表示由顶点
 $i, i+1, \dots, j-1, j$ 组成的凸多边形
 $(i < j)$ 划分后的最小权值。

标记函数: $t[i, j]$ 记录当 $m[i, j]$ 取
 最小值时 i, j 两点所在三角形中第 3 个
 点的位置 k 。

$$\text{递推关系: } m[i, j] = \begin{cases} 0, & i = j-1 \\ \min_{i < k < j} \{ m[i, k] + m[k, j] + d_{ki} + d_{kj} + d_{ij} \}, & i < j-1 \end{cases}$$

满足优化规则

该算法的时间复杂度: $O(n^2 \cdot n) = O(n^3)$

3.15 解: 显然不会出现连续 m 天 ($m \geq 2$) 和在最
 后一天检修的情况, 故可考虑枚举检修日
 来划分子问题。

优化函数: $M[i, j]$ 表示从第 i 天到第 j 天可加工的最大
 任务数 ($1 \leq i \leq j \leq n$)

标记函数: $t[i, j]$ 标记 $M[i, j]$ 取最大值时的检修日
 k , 若不检修则标记为 0

递推关系:

$$M[i, j] = \max \left\{ \sum_{t=i}^j \min \{ x_t, s_t \}, \max_{i < k < j} \{ M[i, k-1] + M[k+1, j] \} \right\}$$

满足优化规则与子问题重叠性

初值: $M[i, i] = \min \{ x_i, s_i \}$

时间复杂度 $O(n^2) \cdot O(n) = O(n^3)$

$$M[j] = \max \left\{ \sum_{t=1}^j \min \{ x_t, s_t \}, \max_{1 \leq k < j} \{ M[k-1] + \sum_{t=k+1}^j \min \{ x_t, s_t - k \} \} \right\}$$

$$M[0] = 0$$

$$O(n^2) \rightarrow O(n^2)$$

3.17 解: 优化函数: $m[i, j]$ 表示从 a_{i1} 到 a_{ij} 的最小和

标记函数: $t[i, j]$ 标记 $m[i, j]$ 取最小值时对应的
 路径中在 $i-1$ 层的结点 ($a_{i-1, j} / a_{i-1, j-1}$)

递推关系: $m[i, j] = \min \{ m[i-1, j-1], m[i-1, j] \} + a_{ij}$

满足子问题重叠性与优化原则 ($2 \leq i \leq j$)

$$t[i, j] = \begin{cases} j-1, & m[i-1, j-1] \leq m[i-1, j] \\ j, & m[i-1, j-1] > m[i-1, j] \end{cases}$$

初值: $m[1, 1] = a_{11}, t[1, 1] = 0$

$$m[i, 1] = m[i-1, 1] + a_{i1}$$

$$m[i, i] = m[i-1, i-1] + a_{ii}$$

结果: $\min_{1 \leq k \leq n} m[n, k]$

时间复杂度为 $O(n^2)$